

Willfly

A neural decision system for new crypto markets

Robinhood Chain first. New launches and spot trading, with selective liquidity provision.

VERSION 2 / 13 SEPTEMBER 2026

Build first: Observatory v0.1. Capture launch and pool events, preserve their history, recover missing data, and inspect what was knowable at each moment. Establish replay and ordinary strategy baselines before evaluating fly-derived neural learning.

This edition: current project direction, training methods, release roadmap, architecture and **77 build tasks** with dependencies and acceptance checks. The original 40-source literature review is retained as a historical appendix.

Evidence boundary: a connectome-derived model can be trained. Its advantage for crypto decisions is unproven. All application tasks are planned; no live integration, model, backtest or funded execution has been validated.

Prepared from the maintained project documents. Estimates and numerical thresholds are proposed engineering choices; provider behavior and exact launch deployments must be validated during Phase 0.

Contents

Current direction and tools

3

Training and hybrid intelligence6

Release roadmap and architecture8

Complete build task catalogue13

P0 - Specification and access

13

P1 - Recorder and recovery

14

P2 - Observatory v0.1

15

P3 - Replay and labels

16

P4 - Baseline policies

18

P5 - Connectome experiment

19

P6 - LP data and LLM integration

20

P7 - Selective LP laboratory

21

P8 - Prospective shadow agent

23

P9 - Funded pilot

24

P10 - Expansion and advanced learning

25

Scientific evaluation design27

Historical literature review30

1. The correct data and software

31

2. What "a real brain" would mean

32

3. What the strongest neuroscience establishes

33

4. Memory, learning and useful computation

34

5. Embodiment: what moving flies demonstrate

35

6. What a DEX liquidity controller could do

36

7. Existing "fly traders" and our proposed design

37

8. A test that can produce a credible answer

38

9. The accounting is part of the experiment

39

10. Implementation path and further experiments

40

Sources and reading coverage

41

Current direction and tools

Updated: 13 September 2026. This document records the user's decisions and research already discussed. No new research was required to make this update.

Objective and scope

Build toward an adaptive agent that discovers and evaluates early crypto opportunities. Start with Robinhood Chain, new token launches, new pairs and memecoins. Spot trading is the initial decision task; selective LP is a distinct candidate action whose economics must be evaluated. Waiting and exiting are first-class actions. Later expansion should reuse data contracts and evaluation while adding chain-specific discovery and execution adapters.

The ambition is strong net financial performance, including with small capital. No guaranteed-return claim is justified. Measure capital recovered after complete entry and exit costs; reported volume, nominal APR and position win rate are insufficient objectives.

The user initially deferred PDFs while brainstorming, then explicitly requested an updated PDF and detailed build roadmap on 13 September 2026. The current PDF now packages this direction, training design, roadmap and complete task backlog. The original LP-range experiment remains useful background but is no longer the primary project plan.

Working scientific thesis

Test whether a connectome-derived recurrent representation improves decisions from sequences of market observations: buyer arrivals, selling pressure, holder changes, liquidity additions/removals and execution conditions. Biological wiring supplies a candidate structural constraint. Market meaning, neural dynamics, training rules and action interfaces are engineered.

A useful target is recognizing when an opportunity changes: initial launch activity becomes sustained demand, liquidity deteriorates despite rising price, or a directional trade becomes a plausible fee-earning opportunity. These are hypotheses, not validated signals. Wallet relationships are clues rather than proof of common ownership or intent.

The proposed neural controller can participate in decision making. It is a computational model using biological measurements, not living tissue or an established uploaded mind. An LLM can interpret text and operate research tools; deterministic software should implement accounting, action validation and eventual transaction mechanics.

Data choice

The existing literature review recommends FAFB with the Shiu implementation as a separate scientific reference and MaleCNS v1.0 as the initial commercially extensible data candidate. Its reviewed release uses CC BY 4.0, while the reviewed FAFB release uses CC BY-NC 4.0. Recheck exact release and dependency licenses before implementation. MaleCNS is not a drop-in reproduction of the FAFB model.

Start with an explicitly documented subnetwork or compact representation if sufficient. Whole-network size is not a success metric. Preserve IDs, graph orientation, filtering, signs, weight transformations and provenance.

Robinhood Chain context

The prior research verified public mainnet documentation for chain ID 4663, native ETH and EVM compatibility. This is distinct from Robinhood's brokerage Crypto API. Mainnet was announced in July 2026. Chainwide activity snapshots supported investigating the ecosystem, but were not independently audited organic-volume measurements or evidence of liquidity in any particular new coin.

- [Official mainnet announcement](#)
- [Official connection documentation](#)
- [DefiLlama chain dashboard](#)

Tools identified with the user

The user's "zenith bo" and "LP agents" were interpreted as the following likely matches. Other scanners were not named. Capabilities below were checked in public documentation, not exercised through authenticated APIs.

Tool	Documented capabilities	Proposed use and limitations
Zenith LP Bot	Telegram LP operations for Robinhood Uniswap V3/V4; user selects pool/range; swap and position opening; public pool explorer, calculator and position viewer	Execution-workflow reference. No documented public automation API was found in the reviewed pages. The provider says it generates and holds wallet keys, with export available.
LP Agent	Portfolio and copy-LP product; authenticated read API for pools and wallet positions; opening-position endpoint explicitly lists Robinhood V3/V4	Candidate LP observation and outcome source. Public transaction-building routes are documented as Solana-only. App support does not establish Robinhood execution API support.
Bitquery	Robinhood pool events, trades and streaming query documentation	Candidate launch/event ingestion. Verify factory coverage, timestamp semantics, retention, delay and commercial terms.
GMGN	Public Robinhood discovery/trading product description; separate agent API documentation	Candidate scanner comparison and enrichment. Robinhood support needs checking for each required API endpoint.
DEX Screener / GeckoTerminal	Pair/token analytics and discovery interfaces	Enrichment and cross-checking candidates; neither newly indexed nor first observed necessarily means newly launched.

Sources:

- [Zenith, public tools, Telegram identity](#).
- [LP Agent, API overview, opening positions, public CLI repository](#).
- [LP Agent documentation index](#). Documentation is inconsistent in places: the overview names V3 while the opening-position endpoint names V3 and V4. Verify live coverage before relying on it.
- [Bitquery Robinhood pool guide](#).
- [GMGN Robinhood product description, agent API](#).
- [DEX Screener API, GeckoTerminal Robinhood pools](#).

Accounting findings

LP Agent's published methodology measures the interval from liquidity addition to removal. Initial acquisition and final conversion can change actual realized outcomes. Our ledger must include those steps, gas, platform charges, failed transactions and residual inventory. Treat vendor PnL as an observation to reconcile, not ground truth for training.

Zenith currently lists \$0.30 per open and per close. Its stated \$0.60 round trip alone equals 3% of a \$20 position or 0.6% of a \$100 position, before other costs. These are arithmetic illustrations, not recommended position sizes or complete cost estimates. Recheck pricing before execution.

Sources: [LP Agent methodology](#), [Zenith pricing and custody](#).

Decisions versus open questions

Decided: Robinhood first; launch discovery and spot decisions first; LP evaluated separately; preserve complete histories including failures; compare biological models with strong ordinary alternatives; update the PDF and detailed build plan; no funded operation now.

Build specification: [roadmap](#) and [task backlog](#). The first implementation target is Observatory v0.1. All 77 application build tasks remain unimplemented; 64 cover the research build and 13 are conditional later work.

Open: exact launchpad/factory and protocol version; data completeness and latency; API access and budget; executable exit modeling; model timescale and topology; training target; capital scenarios; whether fly-derived structure contributes anything beyond the surrounding tools.

No claim of live reliability, profitable learning or biological superiority has been established.

Training and hybrid intelligence

13 September 2026. Targeted research performed after the project-direction and initial experiment-plan updates. Proposed methods have not been implemented or benchmarked here.

Answer

Yes, a model constructed from fly connectivity can be trained. A connectome is an anatomical graph, not a pretrained language model or a financial policy. We must choose mathematical dynamics, inputs, outputs, trainable parameters and feedback. Updating simulated activity is not itself learning; learning changes parameters or another persistent learned representation.

Evidence and boundaries

1. **Task optimization with biological constraints:** Lappalainen et al. trained a fly visual-system model with gradient-based optimization while retaining measured connectivity. Their approximately 45,669 model neurons used 734 shared neural parameters, including neuron time constants, resting potentials and synaptic scale factors; a decoder was trained too. This is direct evidence that connectome constraints and deep learning can coexist. The specialized visual circuit and its parameter-sharing assumptions are not a recipe for a whole-brain market agent. [Nature, 2024](#). Relevant construction and training passages re-read in this session.
2. **Training spiking models:** Neftci, Mostafa and Zenke explain surrogate-gradient methods for learning in spiking networks despite discontinuous spike generation. This establishes a candidate optimization approach, not financial effectiveness or a claim that real fly synapses use backpropagation. [Author paper, 2019](#). Abstract and method framing checked; implementation details need a full-methods review before choosing a spiking learner.
3. **Learned connectomic control:** The FlyGM preprint reports reinforcement-learned locomotion control using a whole-brain graph. The original literature review examined v1 methods; this session checked the v3 abstract, revised June 2026. Do not transfer v1 method details to v3 without comparison. Neither version establishes trading performance. [FlyGM, 2026 preprint](#).
4. **Frozen reservoir:** The original review examined Costi et al.'s connectome-derived reservoir for physical time-series prediction. Its conditional results motivate readout-only training and matched random controls. Full text was previously reviewed; current re-access encountered an access challenge. [Costi et al., 2025](#).

Recommended training progression

Experiment	What changes during training	What it tests
Frozen reservoir	Small output layer; recurrent weights fixed	Whether graph dynamics provide useful temporal features
Constrained tuning	Selected encoder, time constants, gains or synaptic scales, plus output layer	Whether task adaptation improves the biologically structured model
Policy learning	Action policy optimized using a defined replay environment and reward	Whether better predictions become better decisions after costs
Online plasticity	Explicit update rules change allowed parameters from newly available feedback	Adaptation, stability, delayed credit assignment and retention

Select progression by evidence, not model size. A differentiable non-spiking recurrent model is a reasonable initial engineering candidate: spiking is neither necessary for recurrence nor sufficient for biological fidelity. There is no pretrained trading checkpoint here that can simply receive an LLM-style fine-tuning dataset.

For the first task, present market observations only up to time t , preserve recurrent state, predict predeclared future outcomes, and train the readout from labels once their horizon has elapsed. Freeze parameters for evaluation while continuing ordinary activity updates. Record state reset/warm-up rules across assets and episodes. Fit input normalization only on training data. Use identical labels and decision rules across model variants.

Combine tools and LLMs through defined interfaces

Tools: obtain events, quote/execution information, contract facts and LP observations. Preserve source and arrival timestamps. Robinhood execution support must be established separately from data support.

LLM: translate public text into structured, source-linked observations such as narrative category, explicit claims, contradictions and uncertainty. Support research and explanation. Initially use retrieval and schema validation; no LLM fine-tuning is required. Fine-tuning becomes a separate option only if a labeled dataset and evaluation demonstrate a recurring extraction or classification problem.

Numerical neural model: consume market sequences and optional text features, maintain state and estimate outcomes or action preferences. The model need not receive raw prose or call a tool for each neural update.

Policy and execution software: convert outputs into feasible actions, enforce allocation and loss constraints, perform exact accounting and eventually construct transactions. Model output does not override these constraints.

This is modular integration, not a literal merger of an LLM's weights and a fly connectome. Joint training is a possible later research problem, but should not obscure which component creates value.

First hybrid comparison

Run the same chronological experiment with: market features alone; market plus LP behavior; market plus timestamped LLM features; and both additions. Repeat the most promising configuration with ordinary and fly-derived sequence models. This separates data improvements from architectural improvements.

Historical LLM knowledge may leak later token outcomes even when supplied text is timestamped. Mask identities where practical, avoid future retrieval, and use prospective shadow evaluation when contamination cannot be excluded. Do not train on LLM-generated profit labels. Use observed and explicitly modeled outcomes, retaining failed and unresolved exits.

Current recommendation

Start with reliable observations, a complete ledger and strong baseline models. Test a fixed fly-derived reservoir with a trained readout next. Add LP features and LLM interpretation as separate, measured improvements. Only then decide whether internal fine-tuning or reinforcement learning is worth its added complexity.

Release roadmap and architecture

Version 1.0 | 13 September 2026 | Status: planned, implementation not started

What we build first

Willfly Observatory v0.1: a local, read-only Robinhood Chain launch and liquidity recorder with a searchable timeline and a data-quality report. It answers: what appeared, when we learned about it, what happened next, and which evidence is missing?

Its first network adapter captures Uniswap V4 pool initialization and subsequent supported pool activity, preserving pool IDs rather than treating the shared PoolManager as a unique pool. A separate launch adapter records creation events from one verified launchpad version, including launches that never produce a trading pool. The exact launchpad deployment is a Phase 0 decision; no unverified contract address is hard-coded by this plan.

The first useful demo lists newly observed launches and pools, opens a token timeline, shows trades and liquidity changes with source timestamps, marks stale/unknown information, and exports a reproducible dataset. It makes no buy recommendation and requires no funded wallet. The same records later support replay, model training and an evidence-based decision interface.

Deliverable ladder

Release	Outcome	Required phases
v0.1 Observatory	Reliable launch/pool history and inspection UI	P0-P2
v0.2 Trading laboratory	Cost-aware replay, labels and baseline decisions	P3-P4
v0.3 Neural experiment	Measured fly-derived versus ordinary sequence models	P5
v0.4 Hybrid intelligence	Independently measured LP-data and LLM contributions	P6
v0.5 LP laboratory	Selective LP evaluated against spot and idle	P7
v0.6 Shadow agent	Prospective hypothetical actions with live arrival delays	P8; P7 only if LP is enabled
Conditional pilot	Limited funded execution, subject to a separate decision	P9
Conditional expansion	Another chain and more adaptive learning	P10

P0-P8 define the research build. P9-P10 describe contingent work, not current authorization to trade or a commitment to ship it. Every task starts as TODO except tasks explicitly marked CONDITIONAL. Documentation completion does not count as implementation completion.

Phase sequence and gates

Phase	Purpose and exit gate	Effort estimate
P0 Specification and access	Pin deployments and one launch source; prove read access; freeze v0.1 scope and initial experiment assumptions	2-4 engineer-days
P1 Recorder and recovery	Raw capture, canonical decoding, backfill and reorg/restart recovery pass fixture and sampled-chain audits	5-8 days
P2 Observatory	Discovery, timelines, coverage diagnostics and portable dataset export pass the v0.1 demo	4-6 days
P3 Replay and labels	Ledger and fill assumptions reconcile with fixtures and observed transactions; labels expose failures and missing outcomes	6-10 days
P4 Baseline policies	Strong simple and ordinary learned baselines evaluated without leakage; experiments reproducible	4-7 days
P5 Connectome experiment	Graph provenance and numerical stability verified; readout-only training compared with matched controls	6-10 days
P6 LP data and LLM	Additions evaluated separately with cost, latency, provenance and contamination controls	4-7 days
P7 LP policy	Supported fee paths and inventory accounting validated; LP decision adds value or is rejected	7-12 days
P8 Shadow operation	Prospective records demonstrate reliability and quantify difference from replay	3-5 engineering days plus observation window
P9 Funded pilot	Only after explicit capital/execution authorization and successful staged verification	Re-estimate after P8
P10 Expansion and advanced learning	Chain portability and internal tuning tested independently	Re-estimate after evidence

Estimates are planning judgments for one experienced engineer with agent assistance, not measured delivery commitments. P0-P2 total 11-18 engineer-days. P0-P8 total 41-69 engineer-days including LP. Provider access delays, insufficient launch history and observation windows are additional elapsed time. No GPU purchase is assumed. Scope and estimates are reviewed at each gate.

Critical path: P0 -> P1 -> P2 -> P3 -> P4 -> P5 -> P6 -> P8. P7 follows P3/P4 and can be scheduled after the spot shadow milestone if its data is unavailable. Spot research does not wait for LP implementation. These are dependency relationships, not instructions to launch parallel agents.

First build queue

Execute P0-01 through P0-06 before binding the collector to a live deployment. Build P1-01 through P1-08 next, then P2-01 through P2-06. Detailed dependencies, target paths and completion evidence are in the [task backlog](#).

P0 can prepare the repository, configuration schema and offline fixtures while API access is unresolved. Mark fixture runs as synthetic; they cannot satisfy live coverage or latency gates. Production history needs an archive-capable provider. Public rate-limited RPC is sufficient only for appropriately limited connectivity checks, not an assumed production service.

v0.1 demo script

1. Run configuration validation and confirm Robinhood chain identity, source manifest and read-only mode.
2. Start capture for the selected launch source and V4 pools; show a moving checkpoint and arrival-delay histogram.
3. Open a launch that later trades and a launch with no observed trading pool. Explain unknown history rather than deleting the second case.
4. Inspect a pool timeline, including a V4 pool ID, token identifiers, swaps and liquidity changes. Show how each row links to raw evidence.
5. Stop the recorder, replay an overlapping backfill and restart. Logical event counts and canonical history remain consistent.
6. Trigger fixture tests for duplicate delivery, a fork, provider disconnect and malformed events. Show recovery and quarantine output.
7. Export a dataset with configuration, hashes, coverage audit and missing intervals. A second local process reproduces the same logical records.

Proposed v0.1 gate: 72 continuous hours of capture; at least 99.5% discovery recall against an independently fetched canonical log set for the selected contracts and intervals; zero unexplained gaps after recovery; no duplicate logical events; restart/backfill/reorg tests pass. Target p95 observation arrival within 15 seconds of the provider-observed canonical block availability. Record chain timestamp separately. These are engineering targets, not current provider guarantees or proof of adequate latency for profitable trenching. If the comparison source is not independent, label the audit accordingly and keep the gate unresolved.

Engineering decisions

Use a Python-first local research system. Proposed libraries are HTTP/WebSocket clients, typed data validation, Arrow/Parquet, DuckDB, numerical/scientific packages, a compact web API and pytest. Pin actual versions after compatibility checks in P0; this plan does not claim those checks are complete. Add PyTorch only when model work begins. A small server-rendered inspection UI is sufficient for v0.1; a separate frontend stack is optional later.

Store immutable raw batches as compressed JSON lines, analytical partitions as Parquet, and checkpoints/manifests in SQLite. Use DuckDB for local analysis of exported partitions. One process owns metadata writes initially. Introduce distributed queues, hosted databases or GPU infrastructure only when observed load or modeling requirements justify them.

Proposed repository layout

```
src/willfly/
  config/          source manifests, typed settings, experiment specifications
  domain/          events, pool identity, observations, decisions, ledger entries
  adapters/        robinhood_rpc, launchpad, bitquery, lpagent, text_sources
  ingest/          capture, checkpoints, backfill, canonicalization, quarantine
  storage/         raw batches, metadata, parquet exports
  features/        causal windows, token timelines, LP and text features
  replay/          scheduler, quotes, execution assumptions, portfolio ledger
  policies/        baselines, action mapping, constraints
  models/          conventional, reservoirs, connectome, optional tuning
  evaluation/      splits, metrics, uncertainty, ablations, reports
  api/             read-only routes
  ui/             launch list, timeline, quality view, experiment views
  cli.py
configs/          chain, source, recording and experiment manifests
tests/            unit, integration, fixtures, replay and recovery scenarios
data/            ignored local raw/derived datasets
artifacts/        ignored model checkpoints and run outputs
docs/            architecture decisions, runbooks, source matrix and tasks
```

These paths and commands below are specifications; they do not exist as application code yet.

Proposed command contract: willfly doctor, willfly capture, willfly backfill, willfly audit, willfly export, willfly serve, then willfly replay, willfly train, willfly evaluate, and willfly shadow. Each accepts a versioned config path and produces a run ID. Initial releases must contain no transaction-signing or broadcast command.

Data contracts

Record	Required information
RawEvent	chain ID, source, source schema version, block number/hash, parent context, transaction hash, log index, event time, received time, payload, ingestion run, canonical status
PoolIdentity	chain, protocol/version, factory or manager, pool address for V3 or manager plus pool ID for V4, currencies, fee settings, tick spacing, hook identity
Launch	chain, launch contract/version, token identity, creation evidence, creator if observable, first-seen time, origin confidence, linked pools and lifecycle evidence
Observation	token/pool, as-of time, latest included event/arrival time, feature version, values, missingness, quality state and lineage
Decision	run/model/config IDs, observation ID, current inventory, proposed action, constraints applied, reason/evidence references and timing
LedgerEntry	action/transaction ID, asset quantities, cash flows, fees, gas, fill assumptions, marks and timestamps, failure/residual inventory state
ExperimentRun	dataset/code/config hashes, split manifest, seed, model parameters, training budget, metrics, costs, artifacts and decision ledger

Raw event uniqueness includes chain, block hash, transaction hash and log index. Canonical logical projections resolve forks; orphaned evidence remains auditable. Store EVM integer quantities exactly and use explicit decimals. Symbols are display fields, never identity. Native ETH and wrapped ETH remain distinct internally, even if a vendor merges them. Unknown values stay unknown.

Initial experiment specification

Proposed initial decision interval: 15 seconds, using only observations received by that moment. Candidate label horizons: 1, 5 and 15 minutes; the 5-minute outcome is the initial primary target. These are feasibility defaults to freeze or revise at P0/P3 using a calibration sample before the holdout is viewed.

Predict the net result of a fixed-size hypothetical purchase followed by liquidation at the horizon, accompanied by exit availability and adverse-excursion diagnostics. Use exact supported pool mechanics where available; otherwise label execution as modeled/unknown and keep it out of claims of validated profitability. Never infer a filled trade from a quoted midpoint alone.

Research capital scenarios: \$100, \$500 and \$1,000 equivalents, fixed 5% entry allocation, at most three simultaneous positions, no borrowing and no adding to positions in the first comparison. These are controlled simulation parameters, not investment recommendations or authorized spending. Evaluate candidate size relative to executable depth. Use deterministic candidate ranking and ties so simultaneous opportunities obey a single shared cash ledger.

Primary financial metric: paired difference in net portfolio return against the strongest preselected practical baseline, with maximum drawdown and failure rates reported alongside. Initial rejection threshold: a model with worse drawdown by more than 5 percentage points does not advance on return alone. A fly-specific advantage requires a positive paired 95% block-bootstrap interval against both the strong practical and matched nonbiological model across the locked evaluation, and positive excess return in at least three of four chronological test windows. This is a proposed stringent research gate, not a forecast that adequate evidence will be available.

Use at least five initialization seeds for learned candidates and a final untouched chronological holdout. Pre-register bootstrap blocks and group overlapping/correlated launch episodes; do not count multiple seeds as independent markets. If sample size cannot support inference, report inconclusive. Do not fabricate a backtest through sparse history or repeatedly inspect the final holdout.

Observation and deployment gates

P8 initially requires 14 calendar days and at least 200 eligible prospective launch episodes; extend if either is missing. These minimums check operation and provide a research sample, not statistical proof of profitability. Qualification requires no unresolved ledger discrepancy or capture gap, at least 99% scheduled decision availability over healthy-source intervals, and recorded behavior under stale data and outages. Report all excluded intervals explicitly.

At shadow start freeze the model, feature versions, capital rules and quality thresholds. Shadow records hypothetical executions; do not label them fills. LP shadow is enabled only for pool families validated in P7. New strategy versions start a new evaluation run rather than rewriting previous results.

P9 requires a separately agreed chain, assets, signer/custody choice, capital cap, loss limits and permitted actions. It includes unsigned transaction verification, simulation, idempotency, approval limits, receipt reconciliation, pause/recovery and a manual exit procedure. No promise of execution quality follows from a profitable replay.

Tool integration priorities and open gates

P0 validates native RPC and one event provider first. Bitquery is a candidate for enrichment/backfill; the system keeps its own history. LP Agent is a later read adapter, not a Robinhood execution dependency. Zenith is a workflow reference until a supported integration and appropriate custody arrangement are established. LLM providers are selected and versioned in P6, with a request/token budget and a no-LLM fallback.

Source coverage, provider budget and exact launchpad deployment remain open facts. P0-02 evaluates Pons versions and pools.trade as leads already identified in the research, chooses one based on verified creation-event coverage and accessible history, and writes the rationale. If neither can be verified, ship the explicitly labeled pool observatory subset while launch-level v0.1 acceptance remains incomplete. Pool creation is not silently substituted for token launch.

Current sources: [Robinhood connection docs](#), [Bitquery pool events](#), [LP Agent API](#), [Zenith](#). Public documentation was rechecked on 13 September 2026; API behavior and deployment bytecode still need validation.

Build tracking rules

The task catalogue is exhaustive for this roadmap's initial scope, not a prediction of every future implementation issue. New discoveries become explicit tasks linked to a phase. Task IDs are stable. A task is DONE only when its acceptance evidence exists; a phase passes only when its gate is met. Provider-dependent tasks may be BLOCKED without marking offline work complete or misrepresenting coverage. Keep proposed design choices separate from observed results.

Use the [machine-readable backlog](#) and [human-readable backlog](#) together. Effort, progress and phase gates should be updated after each implementation batch. The PDF is a review snapshot; Markdown and the task catalogue are the maintained planning sources.

Complete build task catalogue

64 research-build tasks and 13 conditional follow-on tasks. Each task states what to implement, its dependencies, proposed files and the evidence required for completion. All tasks are currently unimplemented.

P0 - Specification and access

P0-01 - Initialize the research workspace

Status: TODO | **Owner role:** Platform | **Depends on:** None

Target paths: pyproject.toml, src/willfly/, tests/, .gitignore

Create the package, local environment, lockfile, formatting and test commands. Ignore datasets, credentials and model artifacts. Add a tiny offline CLI entry point and document the supported Python version.

Done when: A fresh environment runs the CLI and fixture tests; repository scanning finds no credential values or generated datasets.

P0-02 - Pin network and source deployments

Status: TODO | **Owner role:** Data | **Depends on:** P0-01

Target paths: configs/sources/, docs/source-matrix.md

Verify chain identity, V4 deployment, ABI and creation block from official protocol sources plus chain evidence. Compare Pons versions and pools.trade; select one launch contract/version with creation coverage, including non-graduates. Record provider limits, history and licenses.

Done when: Manifest contains evidence URLs, addresses, ABI hashes, deployment blocks and network checks. At least ten raw examples per supported event family decode; unknown deployment means gate remains open.

P0-03 - Validate read access and provider budget

Status: TODO | **Owner role:** Data | **Depends on:** P0-02

Target paths: src/willfly/adapters/, docs/provider-budget.md

Probe read RPC/archive capability and candidate event API with bounded requests. Measure historical depth, pagination, rate limits, delay and error behavior. Specify daily request/storage estimates and chosen plan without purchasing a subscription.

Done when: Redacted probe report distinguishes tested endpoints from documented ones; a 24-hour sample budget is estimated; unavailable keys are explicit blockers rather than silently mocked results.

P0-04 - Define canonical data contracts

Status: TODO | **Owner role:** Platform | **Depends on:** P0-02

Target paths: src/willfly/domain/, docs/data-contracts.md

Specify raw events, pool identity, launches, observations and lineage. Preserve exact quantities, ETH/WETH identity, V4 pool IDs, timestamps, canonicity and unknown values. Version schemas and reject ambiguous fields.

Done when: Schema fixtures round-trip exact integer quantities and include V3/V4 identity examples, missing metadata and orphaned logs; ambiguous identity fails validation.

P0-05 - Freeze the first experiment proposal

Status: TODO | **Owner role:** Research | **Depends on:** P0-03, P0-04

Target paths: configs/experiments/initial.yaml, docs/evaluation-contract.md

Record eligibility, decision interval, target horizons, capital scenarios, splits, drawdown constraints, inference rules and data-quality exclusions. Reserve a calibration sample and untouched holdout; document changes before holdout access.

Done when: Versioned specification includes every metric and gate from the roadmap, known feasibility dependencies and a deterministic split construction rule; no test result is used for specification selection.

P0-06 - Create offline fixtures and v0.1 gate

Status: TODO | **Owner role:** QA | **Depends on:** P0-04

Target paths: tests/fixtures/, docs/runbooks/v0.1.md

Prepare provenance-tagged observed fixtures where available and explicitly synthetic failure cases. Encode the seven-step observatory demo and distinguish synthetic verification from live evidence.

Done when: Fixture manifest identifies origin and hashes; v0.1 checklist covers disconnected source, duplicate delivery, malformed log, native asset and fork cases.

P1 - Recorder and recovery

P1-01 - Implement raw storage and metadata

Status: TODO | **Owner role:** Data | **Depends on:** P0-01, P0-04

Target paths: src/willfly/storage/

Write compressed append-only raw batches with hashes and a transactional SQLite checkpoint/manifest store. Define partition naming and atomic batch publication.

Done when: Interruption before and after checkpoint commit does not lose acknowledged events; corrupt/truncated batches are detected and recoverable from the last good checkpoint.

P1-02 - Implement Robinhood log capture

Status: TODO | **Owner role:** Data | **Depends on:** P0-03, P0-06, P1-01

Target paths: src/willfly/adapters/robinhood_rpc.py, src/willfly/ingest/capture.py

Capture configured contracts through supported subscriptions or bounded polling, recording arrival time and chain context. Keep subscription mechanics distinct from the sequencer feed; enforce read-only operation.

Done when: Recorded event batches match sample RPC receipts; throttling/backoff works; wrong-chain endpoint is rejected; no signing or transaction-send method is exposed.

P1-03 - Decode pool and trading events

Status: TODO | **Owner role:** Data | **Depends on:** P1-02

Target paths: src/willfly/adapters/protocols/

Decode initialization, swaps and liquidity changes for supported V4 paths. Implement exact currency ordering and pool identity; flag unsupported hooks instead of treating all V4 behavior as standard.

Done when: Known receipts decode deterministically with raw lineage; two pools sharing a manager remain separate; malformed and unsupported events enter quarantine.

P1-04 - Decode one launchpad lifecycle

Status: TODO | **Owner role:** Data | **Depends on:** P0-02, P1-02

Target paths: src/willfly/adapters/launchpad.py

Implement the selected version's creation and lifecycle events. Link pools only using verifiable evidence; retain launches without pool creation and separate first seen from creation time.

Done when: Fixtures cover trading and non-trading launches, late discovery and version mismatch; origin confidence and unknown history are visible.

P1-05 - Implement resumable backfill

Status: TODO | **Owner role:** Data | **Depends on:** P1-03, P1-04

Target paths: src/willfly/ingest/backfill.py

Fetch bounded historical ranges with adaptive page sizes, pagination completeness checks and checkpoints. Merge overlap with live capture without losing provenance.

Done when: Stopping mid-range and resuming yields the same logical records as uninterrupted replay; truncated pages and provider range limits are surfaced.

P1-06 - Implement canonicalization and fork recovery

Status: TODO | **Owner role:** Data | **Depends on:** P1-05

Target paths: src/willfly/ingest/canonicalize.py

Track block hashes/parents and provisional versus confirmed projections using the verified network policy. Roll back derived state on fork changes, retaining raw orphan evidence.

Done when: A multi-block fork fixture produces the expected canonical dataset and repaired checkpoints; orphaned events never remain in canonical counts or training snapshots.

P1-07 - Add supervision and quality telemetry

Status: TODO | **Owner role:** Platform | **Depends on:** P1-02, P1-06

Target paths: src/willfly/ingest/supervisor.py, src/willfly/ingest/quality.py

Measure arrival delay, block lag, gap ranges, quarantine counts, retries and storage growth. Add reconnect, bounded retry and disk-space behavior; redact credentials from logs.

Done when: Disconnect/reconnect and storage-failure tests produce explicit degraded states and successful gap recovery; health cannot report green while a gap is unresolved.

P1-08 - Run recorder coverage and recovery audit

Status: TODO | **Owner role:** QA | **Depends on:** P1-03, P1-04, P1-05, P1-06, P1-07

Target paths: src/willfly/evaluation/coverage.py, artifacts/coverage/

Compare configured source intervals with separately fetched canonical logs. Run duplicate, crash and fork drills. Record numerator/denominator, provider independence and exclusions.

Done when: v0.1 capture gate is measured over 72 hours: discovery recall target, gap accounting and recovery checks reported. Any unmet condition remains an explicit gate failure.

P2 - Observatory v0.1

P2-01 - Build discovery and lifecycle projections

Status: TODO | **Owner role:** Data | **Depends on:** P1-08

Target paths: src/willfly/features/discovery.py

Materialize launches, pools, trading status and evidence-backed lifecycle transitions. Keep inactive launches, pools without known launch origin and late-indexed records.

Done when: Counts reconcile to raw canonical events; launch time, pool time and first-observed time remain separate and queryable.

P2-02 - Build causal token timelines

Status: TODO | **Owner role:** Data | **Depends on:** P2-01

Target paths: src/willfly/features/timelines.py

Calculate trailing trades, flow and liquidity summaries using event and arrival cutoffs. Include available wallet aggregates, missingness and staleness. Do not claim full holder coverage from partial transfer history.

Done when: Appending future data cannot change an earlier as-of snapshot except through a new explicitly versioned correction; missing history produces unknown features.

P2-03 - Implement portable dataset export

Status: TODO | **Owner role:** Data | **Depends on:** P2-02

Target paths: src/willfly/storage/export.py

Export canonical Parquet partitions plus schema, source/config hashes, gaps and quality report. Keep API-specific payloads traceable but outside model contracts.

Done when: Re-import reproduces row counts, logical keys and feature hashes; large integer identities and quantities survive round-trip.

P2-04 - Expose read-only inspection API and CLI

Status: TODO | **Owner role:** Platform | **Depends on:** P2-03

Target paths: src/willfly/api/, src/willfly/cli.py

Implement doctor, capture, backfill, audit, export and serve commands. Provide paginated launches, token timeline, pool and health endpoints, bound locally by default.

Done when: CLI commands produce run IDs and useful exit codes; endpoint integration tests cover pagination, invalid identity and degraded data. No trading route exists.

P2-05 - Build the observatory interface

Status: TODO | **Owner role:** Product | **Depends on:** P2-04

Target paths: src/willfly/ui/

Create launch list, token timeline and quality views. Show source time, arrival time, stale/missing fields and explorer evidence links. Add filters for lifecycle and supported protocol.

Done when: The demo can inspect one active launch and one non-graduate, trace an event to evidence and identify a data outage without reading logs.

P2-06 - Package and accept Observatory v0.1

Status: TODO | **Owner role:** QA | **Depends on:** P2-05, P1-08

Target paths: docs/runbooks/observatory.md, artifacts/releases/v0.1/

Document install, source setup, capture/restart, audit/export and recovery. Run the full v0.1 demo from a clean local environment and publish a release evidence bundle.

Done when: Another local process follows the runbook and reproduces the export; phase gates pass or the release is explicitly labeled a partial pool-only prototype.

P3 - Replay and labels

P3-01 - Implement a deterministic replay scheduler

Status: TODO | **Owner role:** Research | **Depends on:** P2-06

Target paths: src/willfly/replay/scheduler.py

Replay by observed availability, with event ordering, strategy ticks and execution delay. Separate historical event timestamps from reconstructed availability when actual arrival data is unavailable.

Done when: Repeated fixture runs produce identical observations and action ordering; the strategy cannot inspect future events or corrected data before availability.

P3-02 - Implement the portfolio ledger

Status: TODO | **Owner role:** Quant | **Depends on:** P3-01

Target paths: src/willfly/replay/ledger.py

Track exact asset balances, trades, gas, protocol/platform fees, marks and residual inventory. Separate realized cash flows from modeled liquidation value and include failed actions.

Done when: Asset conservation and hand-calculated multi-action fixtures reconcile; deposits cannot be counted as profit and fees are not deducted twice.

P3-03 - Implement supported quotes and fill assumptions

Status: TODO | **Owner role:** Quant | **Depends on:** P3-02

Target paths: src/willfly/replay/execution.py

Model supported pool mechanics, transaction delay and costs at the chosen capital sizes. Use archive state where available; label approximations. Exclude unsupported hooks from executable claims while retaining their observed episodes.

Done when: Known transaction examples reconcile within declared tolerances; no route, shallow depth, revert, missing state and price changes generate explicit outcomes.

P3-04 - Implement causal training labels

Status: TODO | **Owner role:** Research | **Depends on:** P3-03, P0-05

Target paths: src/willfly/features/labels.py

Generate 1/5/15-minute outcome labels with a primary 5-minute task after calibration. Record entry/exit feasibility, net returns and adverse excursions. Distinguish unresolved observations from actual losses.

Done when: Every label links to its horizon and fill assumptions; failed and unsellable episodes remain represented; incomplete horizons are marked censored.

P3-05 - Implement portfolio constraints and candidate allocation

Status: TODO | **Owner role:** Quant | **Depends on:** P3-02, P0-05

Target paths: src/willfly/policies/constraints.py

Enforce simulation capital, fixed entry allocation, concurrent-position limit, no leverage and no adding. Resolve simultaneous candidates deterministically against one shared cash ledger.

Done when: Competing opportunities cannot spend the same cash; rejected actions carry reasons; fixtures exercise all caps and ties.

P3-06 - Implement leakage-resistant split manifests

Status: TODO | **Owner role:** Research | **Depends on:** P3-04

Target paths: src/willfly/evaluation/splits.py

Create chronological calibration/train/validation/test manifests, purge overlapping label horizons and group related episodes where justified. Fit normalization only on training data.

Done when: Automated checks reject overlapping labels and future-derived features; held-out partitions are immutable and their hashes recorded before tuning.

P3-07 - Audit replay mechanics and cost stress

Status: TODO | **Owner role:** QA | **Depends on:** P3-03, P3-04, P3-05, P3-06

Target paths: tests/replay/, artifacts/replay-audit/

Reconcile at least 20 observed transaction examples spanning supported action paths, plus adverse synthetic fixtures. Evaluate increased delay, fees, slippage and incomplete liquidity.

Done when: An audit lists exact versus approximate paths, tolerances and residual discrepancies; unsupported paths cannot silently pass as validated fills.

P3-08 - Publish the replay dataset and evidence bundle

Status: TODO | **Owner role:** Research | **Depends on:** P3-07

Target paths: artifacts/datasets/, docs/runbooks/replay.md

Package dataset, label/split configs, accounting evidence and reproduction commands. Define a minimum adequate sample after observed episode counts and correlation analysis.

Done when: An independent local rerun reconstructs labels and ledgers; insufficient sample or historical state is explicitly inconclusive and blocks profitability claims.

P4 - Baseline policies

P4-01 - Implement simple baseline strategies

Status: TODO | **Owner role:** Quant | **Depends on:** P3-08

Target paths: src/willfly/policies/baselines.py

Implement idle cash, fixed-horizon hold, deterministic confirmation/momentum and a simple flow/liquidity filter with predeclared parameters and shared constraints.

Done when: Every strategy uses identical information cutoffs and costs; a fixture demonstrates the intended trigger and actual allocation behavior.

P4-02 - Implement conventional prediction baselines

Status: TODO | **Owner role:** Research | **Depends on:** P3-08

Target paths: src/willfly/models/conventional.py

Train regularized raw-feature models and one compact ordinary recurrent model. Use the same target family and feature information as the proposed neural experiment.

Done when: Training is reproducible from hashes and seeds; recurrent state cannot leak between unrelated token episodes or across split boundaries.

P4-03 - Define common prediction-to-action mapping

Status: TODO | **Owner role:** Quant | **Depends on:** P4-01, P4-02

Target paths: src/willfly/policies/prediction.py

Map predictions and uncertainty to enter/hold/exit decisions under shared fixed-size rules. Select thresholds only on validation and retain watch as an action.

Done when: Both learned model families run under the same decision rule; changing model predictions cannot bypass constraints.

P4-04 - Implement experiment tracking and budgets

Status: TODO | **Owner role:** Platform | **Depends on:** P4-02

Target paths: src/willfly/evaluation/runs.py

Record source/code/data/config hashes, seeds, training time, parameter counts, search trials and artifacts. Enforce comparable tuning opportunities and log failed runs.

Done when: A run can be reconstructed without undocumented manual settings; search-budget overrun is detected and failed models remain in the experiment record.

P4-05 - Implement metrics and paired uncertainty

Status: TODO | **Owner role:** Research | **Depends on:** P4-03, P4-04

Target paths: src/willfly/evaluation/metrics.py

Calculate net and excess return, drawdown, tail outcomes, failure rates, turnover, calibration and exposure. Implement paired block-bootstrap with episode grouping and locked resampling rules.

Done when: Hand-calculated fixtures verify metrics; seed repetitions are not treated as independent markets; insufficient independent blocks are flagged.

P4-06 - Run walk-forward evaluation and stresses

Status: TODO | **Owner role:** Research | **Depends on:** P4-05

Target paths: artifacts/baselines/

Tune on permitted windows, freeze candidates, then evaluate locked windows with cost and delay stresses. Record each regime rather than selecting only favorable launches.

Done when: All predeclared periods and capital scenarios appear; the strongest practical baseline is selected before the neural holdout comparison.

P4-07 - Accept baseline laboratory v0.2

Status: TODO | **Owner role:** QA | **Depends on:** P4-06

Target paths: docs/results/baseline-review.md

Review accounting, sample adequacy, inference and reproducibility. Document whether any candidate shows net opportunity and which data limits remain.

Done when: Gate report states pass, revise or inconclusive; no positive narrative substitutes for missing costs, failures or holdout evidence.

P5 - Connectome experiment

P5-01 - Pin and validate connectome data

Status: TODO | **Owner role:** Research | **Depends on:** P4-07

Target paths: configs/connectome/, src/willfly/models/connectome/ingest.py

Select exact MaleCNS release and review attribution/license terms and dependencies. Download only needed tables; hash and validate IDs, annotations and curated-neuron filters.

Done when: Manifest links every input to its release and license; no floating-point ID conversion; counts and excluded records reconcile.

P5-02 - Build the sparse graph and controls

Status: TODO | **Owner role:** Research | **Depends on:** P5-01

Target paths: src/willfly/models/connectome/graph.py

Specify subnetwork selection, orientation, aggregation, sign assumptions and weight transforms. Build matched random and shuffled controls retaining declared degree/sign/weight properties where feasible.

Done when: Toy directed graph verifies orientation; graph statistics and matching deviations are reported; controls differ only in documented properties.

P5-03 - Implement stateful reservoir dynamics

Status: TODO | **Owner role:** Research | **Depends on:** P5-02

Target paths: src/willfly/models/reservoir.py

Use a compact rate-based recurrent model initially, with explicit decay, input scale and stability controls. Preserve state per episode and specify warm-up/reset behavior. Benchmark sparse memory and runtime.

Done when: Impulse and constant-input tests expose decay and stability; no accidental dense full-brain matrix allocation; model remains finite on training input stress.

P5-04 - Train the frozen-reservoir readout

Status: TODO | **Owner role:** Research | **Depends on:** P5-03, P4-04

Target paths: src/willfly/models/readout.py

Train a regularized output layer on reservoir activity with identical labels and split rules. Tune only declared readout/reservoir hyperparameters; keep recurrent weights fixed during training.

Done when: Before/after graph hashes match; readout parameters change; held-out prediction generation uses only frozen training artifacts and causal activity.

P5-05 - Run matched model comparisons

Status: TODO | **Owner role:** Research | **Depends on:** P5-04, P4-06

Target paths: artifacts/neural/comparisons/

Compare fly graph, random reservoir, shuffled graph, raw-feature readout and ordinary recurrent model with at least five seeds and comparable budgets.

Done when: Same dataset, policy and ledger used throughout; performance, compute and uncertainty are reported for every predeclared comparator.

P5-06 - Run contribution and state ablations

Status: TODO | **Owner role:** Research | **Depends on:** P5-05

Target paths: src/willfly/evaluation/ablations.py

Reset state, remove the reservoir, perturb input mapping and graph organization. Preserve fair retuning rules and report each intervention's scope.

Done when: Results distinguish useful recurrence, feature effects and biological topology; conclusions cannot attribute surviving wrapper performance to the removed component.

P5-07 - Apply neural progression gate

Status: TODO | **Owner role:** Research | **Depends on:** P5-06

Target paths: docs/results/neural-review.md

Apply predeclared return, drawdown, multi-window and uncertainty criteria. Assess sensitivity to capital, delay and graph assumptions; report negative and inconclusive results.

Done when: Signed-off research note chooses retain, revise or reject the fly component with traceable evidence; internal fine-tuning is not automatically triggered.

P5-08 - Package neural experiment v0.3

Status: TODO | **Owner role:** Platform | **Depends on:** P5-07

Target paths: artifacts/models/, docs/runbooks/neural.md

Save graph manifest, model/readout state, feature config, model card and inference interface. Document data licenses and limits, including lack of demonstrated biological fidelity in the market task.

Done when: Reloaded model reproduces reference predictions and action ledger within declared numerical tolerance on the same environment.

P6 - LP data and LLM integration

P6-01 - Validate LP Agent read coverage

Status: TODO | **Owner role:** Data | **Depends on:** P4-07

Target paths: src/willfly/adapters/lpagent.py, docs/integrations/lpagent.md

Probe Robinhood V3/V4 read endpoints, authentication, paging, freshness and field meanings. Reconcile native ETH/WETH handling and vendor PnL against raw evidence.

Done when: Endpoint matrix separates tested, missing and unsupported features; Solana-only transaction routes are not presented as Robinhood execution support.

P6-02 - Build causal LP behavior features

Status: TODO | **Owner role:** Research | **Depends on:** P6-01

Target paths: src/willfly/features/lp_behavior.py

Record wallet position changes, range movement and liquidity withdrawal with arrival times. Avoid selecting wallets based on future leaderboard outcomes; express incomplete coverage.

Done when: Feature fixtures prevent future ranking leakage; wallet identities are observations rather than unverified claims of skill or common ownership.

P6-03 - Define text evidence and LLM extraction

Status: TODO | **Owner role:** Research | **Depends on:** P4-07

Target paths: src/willfly/adapters/text_sources.py, src/willfly/features/text_schema.py

Choose accessible public sources and a versioned extraction schema for claims, narrative, contradictions, uncertainty and source spans. Archive only permitted content with publication and retrieval times.

Done when: A labeled calibration set checks extraction accuracy and unsupported claims; missing publication time or evidence is explicit rather than fabricated.

P6-04 - Implement bounded LLM enrichment

Status: TODO | **Owner role:** Platform | **Depends on:** P6-03

Target paths: src/willfly/features/llm.py

Use retrieval, schema validation, caching, retries and per-run request/token budgets. Treat external content as untrusted data. Record model/prompt versions; provide a no-LLM fallback.

Done when: Invalid outputs, hostile instructions, timeouts and budget exhaustion cannot change permissions or block the numerical decision path; source-backed output survives schema checks.

P6-05 - Audit historical contamination

Status: TODO | **Owner role:** Research | **Depends on:** P6-04, P3-06

Target paths: docs/results/llm-contamination.md

Evaluate identity masking and time-bounded retrieval. Separate extraction labels from market outcomes; do not use present-day model knowledge as historical evidence.

Done when: Audit identifies usable retrospective features and those requiring prospective evaluation; no LLM-created profit labels enter training.

P6-06 - Run feature and architecture factorial tests

Status: TODO | **Owner role:** Research | **Depends on:** P5-08, P6-02, P6-05

Target paths: artifacts/hybrid/

Compare market-only, plus LP, plus text, and both feature sets across ordinary and fly-derived candidates. Charge data/LLM costs and measure added delay.

Done when: Marginal contributions are reported separately; a benefit from additional data is not misattributed to biological wiring; invalid historical LLM tests are marked unresolved.

P6-07 - Package evidence-backed decision inspection

Status: TODO | **Owner role:** Product | **Depends on:** P6-06, P2-05

Target paths: src/willfly/ui/decisions/, docs/results/hybrid-review.md

Show observations, predictions, constraint outcomes and source citations for recorded decisions. Use LLM prose only to summarize recorded evidence, with uncertainty visible.

Done when: A reviewer can reproduce why an action was permitted and inspect evidence; explanations do not claim hidden neural motives or unverified causal intent.

P7 - Selective LP laboratory

P7-01 - Select and verify an LP pool family

Status: TODO | **Owner role:** Quant | **Depends on:** P3-08, P4-07

Target paths: configs/lp/, docs/lp-scope.md

Choose one supported V3 or V4 family from verified deployment and hook behavior. Document position contracts, fee rules, supported actions, required historical state and exit constraints.

Done when: Source/bytecode evidence and observed receipts support the selected mechanics; unknown hooks or unavailable state block that family rather than being approximated as standard.

P7-02 - Implement position and fee replay

Status: TODO | **Owner role:** Quant | **Depends on:** P7-01

Target paths: src/willfly/replay/lp_positions.py

Model tick rounding, active liquidity, fee growth, range crossing and token quantities. State the small-position counterfactual assumption and its capacity limits.

Done when: Hand-calculated positions and observed checkpoints reconcile; out-of-range positions accrue only fees justified by the actual supported path.

P7-03 - Implement LP lifecycle accounting

Status: TODO | **Owner role:** Quant | **Depends on:** P7-02, P3-02

Target paths: src/willfly/replay/lp_execution.py

Account for acquisition, opening, fee collection, resizing, removal and final conversion, including gas/platform costs and residual tokens. Model failure and unsupported exit behavior.

Done when: Complete base-asset-to-base-asset cycles reconcile; displayed LP PnL is not substituted for portfolio profit; IL/LVR diagnostics are not double-subtracted.

P7-04 - Build restricted LP baseline policies

Status: TODO | **Owner role:** Quant | **Depends on:** P7-03

Target paths: src/willfly/policies/lp_baselines.py

Compare fixed wide range, a simple volatility-based range and idle/spot alternatives. Initially use fixed allocation and a small discrete range set with minimum dwell time.

Done when: All policies share starting capital and recorded price exposure benchmarks; high nominal APR alone cannot trigger a favorable evaluation.

P7-05 - Add LP as a competing policy action

Status: TODO | **Owner role:** Research | **Depends on:** P7-04, P6-07

Target paths: src/willfly/policies/mode_selection.py

Allow spot, selected LP or idle based on the same information cutoff and shared portfolio. Charge mode-switching costs and enforce supported-pool constraints.

Done when: Mode changes preserve inventory and cash; unsupported pools cannot receive LP actions; no double allocation across spot and LP engines.

P7-06 - Stress LP economics and counterfactuals

Status: TODO | **Owner role:** Quant | **Depends on:** P7-05

Target paths: artifacts/lp/stress/

Evaluate directional moves, depleted liquidity, range churn, higher fees/delay, hook-related exit costs and greater position sizes. Separate observed from modeled behavior.

Done when: Report includes failure and sensitivity cases plus capacity limits; performance that disappears under plausible costs is rejected or labeled inconclusive.

P7-07 - Audit LP replay and decisions

Status: TODO | **Owner role:** QA | **Depends on:** P7-06

Target paths: tests/lp/, docs/results/lp-audit.md

Reconcile sampled historical positions and complete lifecycle fixtures; inspect contract-specific exceptional paths and fee attribution independently of the model.

Done when: No unresolved accounting discrepancy in supported cases; unsupported behavior is documented and excluded from claimed execution coverage.

P7-08 - Decide LP eligibility for shadow

Status: TODO | **Owner role:** Research | **Depends on:** P7-07

Target paths: docs/results/lp-review.md

Compare selective LP with spot and idle under the registered metric, exposure and drawdown rules. Publish the outcome even if LP adds no value.

Done when: Gate explicitly enables named LP families or leaves LP disabled; spot shadow remains independently eligible.

P8 - Prospective shadow agent

P8-01 - Freeze a prospective shadow configuration

Status: TODO | **Owner role:** Research | **Depends on:** P6-07

Target paths: configs/shadow/, docs/shadow-protocol.md

Select the best evidence-supported model, which may be ordinary, and freeze feature/model/source versions and capital rules. Enable LP only after P7-08.

Done when: Config hash and start time are recorded before observation; study requires 14 days and 200 eligible launches, with extensions for inadequate coverage.

P8-02 - Implement live hypothetical decision loop

Status: TODO | **Owner role:** Platform | **Depends on:** P8-01

Target paths: src/willfly/shadow/runner.py

Use actual arrival timestamps, shared portfolio constraints and modeled execution to record proposed actions. Keep signing/broadcast absent; persist decision checkpoints.

Done when: Shadow runs across restarts without duplicate hypothetical orders; each record is labeled hypothetical and traceable to received observations.

P8-03 - Implement degraded-mode behavior

Status: TODO | **Owner role:** Platform | **Depends on:** P8-02

Target paths: src/willfly/shadow/health.py

On stale or contradictory data, block new hypothetical entries and record the prescribed management behavior for existing simulated positions. Track decision availability and missed deadlines.

Done when: Outage drills show explicit pause/degradation and recovery; unknown execution state is never reported as a successful exit.

P8-04 - Run the prospective observation window

Status: TODO | **Owner role:** Research | **Depends on:** P8-03

Target paths: artifacts/shadow/

Collect the frozen study, including adverse episodes and all excluded intervals. Track latency, decision availability, provider costs and modeled performance.

Done when: Minimum duration/episode requirements and 99% healthy-interval decision availability are measured; unmet targets or source gaps remain visible.

P8-05 - Reconcile shadow against replay assumptions

Status: TODO | **Owner role:** Quant | **Depends on:** P8-04

Target paths: docs/results/shadow-review.md

Compare live availability, quotes and hypothetical fills against historical assumptions; quantify drift, missed opportunities and data revisions.

Done when: Evidence explains material differences and applies cost stresses; prospective results are not represented as funded returns.

P8-06 - Publish the research go/no-go package

Status: TODO | **Owner role:** Research | **Depends on:** P8-05

Target paths: docs/releases/v0.6.md

Summarize all model comparisons, costs, data quality, uncertainty and open execution issues. Recommend continued shadow, revised experiments or a separately scoped pilot.

Done when: Reproducible bundle and decision record exist; no funded deployment is inferred from a positive report.

P9 - Funded pilot

P9-01 - Specify authorized pilot boundaries

Status: CONDITIONAL | **Owner role:** Product | **Depends on:** P8-06

Target paths: docs/pilot-scope.md, configs/pilot/

Only after a separate user decision, record chain, allowed assets/protocols, signer/custody, funded capital cap, daily loss limit, position cap and allowed transaction types.

Done when: Exact authorization and config match; no placeholder capital limits or blanket unlimited permissions remain.

P9-02 - Implement isolated signer interface

Status: CONDITIONAL | **Owner role:** Security | **Depends on:** P9-01

Target paths: src/willfly/execution/signer.py

Select the approved custody mechanism and separate proposal, validation and signing. Keep private keys out of model inputs, logs and repository files.

Done when: Signer contract tests reject unapproved destinations/actions and prevent the LLM or market content from expanding authority.

P9-03 - Implement transaction construction and simulation

Status: CONDITIONAL | **Owner role:** Execution | **Depends on:** P9-02

Target paths: src/willfly/execution/transactions.py

Build approved swaps and, if enabled, LP calls with exact amounts, slippage/deadlines and bounded allowances. Simulate against current state and inspect calldata.

Done when: Fixtures and fork/test environment runs reconcile decoded intent with proposed action; failed simulation blocks submission and does not count as a fill.

P9-04 - Implement submission and receipt reconciliation

Status: CONDITIONAL | **Owner role:** Execution | **Depends on:** P9-03

Target paths: src/willfly/execution/submission.py

Handle nonces, idempotency, replacement, timeouts, reverts and confirmed receipts. Track pending transaction exposure and avoid unsafe resubmission after uncertain responses.

Done when: Recovery tests demonstrate no duplicate economic action and correct ledger state for pending, replaced, reverted and finalized transactions.

P9-05 - Verify pause, withdrawal and recovery procedures

Status: CONDITIONAL | **Owner role:** Security | **Depends on:** P9-04

Target paths: docs/runbooks/pilot-recovery.md, tests/execution/

Test limit enforcement, emergency pause, approval management, manual inspection and permitted exit procedures. Document dependencies on contract liquidity and token transfer behavior.

Done when: Drills demonstrate signer/ledger consistency and bounded permissions; recovery instructions distinguish a possible action from a guaranteed exit.

P9-06 - Run the separately authorized bounded pilot

Status: CONDITIONAL | **Owner role:** Execution | **Depends on:** P9-05

Target paths: artifacts/pilot/

Start only inside the agreed limits. Monitor receipts, costs and drawdown; stop according to the frozen pilot rules and record every funded action.

Done when: Actual wallet balances reconcile with the ledger; limit breaches pause the system; no automatic capital increase occurs.

P9-07 - Review funded results before scaling

Status: CONDITIONAL | **Owner role:** Research | **Depends on:** P9-06

Target paths: docs/results/pilot-review.md

Compare actual execution and returns with shadow, including failed transactions and residual positions. Assess whether any demonstrated edge survives all paid costs.

Done when: Complete cash-flow report supports stop, revise or separately authorized continuation; scaling is not triggered by isolated wins.

P10 - Expansion and advanced learning

P10-01 - Implement a second-chain data adapter

Status: CONDITIONAL | **Owner role:** Data | **Depends on:** P8-06

Target paths: src/willfly/adapters/second_chain/

After selecting a named market, map its identity, finality, events and pool mechanics into existing contracts. Revalidate provider coverage and source licenses.

Done when: Cross-chain conformance fixtures pass; chain-specific facts are not hidden behind Robinhood assumptions.

P10-02 - Rebuild cross-market evaluation

Status: CONDITIONAL | **Owner role:** Research | **Depends on:** P10-01

Target paths: configs/experiments/cross_chain/

Measure distribution shift and define fresh chronological holdouts before retraining. Compare pooled and per-chain models with matched costs and budgets.

Done when: Results distinguish transfer from retraining; Robinhood performance is not treated as evidence on the new chain.

P10-03 - Test constrained internal fine-tuning

Status: CONDITIONAL | **Owner role:** Research | **Depends on:** P5-07

Target paths: src/willfly/models/connectome/tuning.py

Only if representation evidence justifies it, tune a limited encoder, gains/time constants or synaptic scales. Document graph/sign constraints and use frozen-reservoir and ordinary trained controls.

Done when: Gradient/parameter-change tests and held-out comparisons show whether extra tuning helps; biological fidelity claims remain distinct from financial usefulness.

P10-04 - Test offline policy learning

Status: CONDITIONAL | **Owner role:** Research | **Depends on:** P3-08, P4-07

Target paths: src/willfly/models/policy_learning.py

If replay supports the required counterfactuals, compare imitation or offline RL with the existing prediction-to-action policy. Account for missing actions and expert-wallet selection bias.

Done when: Evaluation exposes unsupported action regions and reward shortcuts; apparent gain cannot rely on unmodeled fills or privileged demonstrator information.

P10-05 - Test online adaptation in simulation or shadow

Status: CONDITIONAL | **Owner role:** Research | **Depends on:** P10-03, P8-06

Target paths: src/willfly/models/plasticity.py

Specify eligibility traces, delayed feedback, update bounds, retention tests and rollback checkpoints. Start with simulated or hypothetical outcomes only.

Done when: Frozen-learning, shuffled-feedback and regime-shift controls show adaptation and retention; no live funded self-modification is enabled.

P10-06 - Review infrastructure and broader product scope

Status: CONDITIONAL | **Owner role:** Platform | **Depends on:** P10-02

Target paths: docs/scaling-review.md

Use measured storage, latency and compute demands to decide whether hosted services, queues, GPU training or a richer UI are justified. Re-estimate the next roadmap.

Done when: Capacity report and proposed cost budget support each infrastructure change; broader markets and funded authority remain explicitly scoped.

Scientific evaluation design

Date: 13 September 2026. Planning document, not implementation status or evidence of performance.

The detailed implementation sequence, numerical feasibility defaults and phase gates are now specified in the [build roadmap](#), with individual work items in the [77-task backlog](#). This document retains the scientific design; use the roadmap for build order.

Primary question

Does a fly-derived recurrent model improve new-token decisions after realistic costs, compared with simple strategies and ordinary machine learning given the same information?

Secondary question: does liquidity-provider behavior add predictive value beyond token price, trades and holder features? Evaluate this separately so an improvement from LP data is not incorrectly attributed to the connectome.

Proposed system

1. **Observation adapters:** chain events and scanner APIs, with both event time and actual arrival time. Persist raw evidence, source identity, factory/version and missingness.
2. **Feature pipeline:** numerical market sequences and optional structured text features. No future data or hidden action recommendation in the encoder.
3. **Decision candidates:** simple rules, conventional learned models, random reservoirs and the fly-derived reservoir, all using comparable information and tuning budgets.
4. **Policy and constraints:** map outputs to watch/enter/hold/reduce/exit initially. Add LP and sizing only after the restricted task is evaluable. Validate action feasibility independently of the model.
5. **Replay and ledger:** simulate delayed fills, costs and exits; reconcile every action with portfolio balances.
6. **Research interface:** an LLM retrieves evidence, summarizes narratives, explains recorded decisions and supports experiment analysis. Market content is untrusted input, never authority to change wallet permissions or system rules.

The initial model should operate without an LLM in its critical path. Add timestamped LLM features as a measured intervention. Natural-language explanations must refer to actual evidence and model outputs rather than inventing a neural rationale.

How training could work

A. Train a readout first

Keep the chosen recurrent graph fixed. Encode past market observations into activity; train a small output layer on historical outcomes. This is training a decision interface, not fine-tuning a pretrained financial brain. A static graph does not come with financial knowledge or an intrinsic learning rule.

Candidate targets include executable forward returns, adverse excursion, exit failure risk and liquidity deterioration over predeclared horizons. Missing or impossible exits must not silently disappear from labels. Start with one target family and one fixed sizing policy, selected after data feasibility is known.

B. Train constrained parameters later

If A shows useful representation, train a limited input encoder, neuron time constants/gains, edge scaling or selected synaptic weights. Preserve graph masks or signs only where scientifically justified, and record every assumption. Differentiable rate models can use ordinary gradients; spiking models need an appropriate method such as surrogate gradients or derivative-free optimization. These are candidate engineering approaches pending targeted source verification.

Changing the model to fit markets reduces the strength of claims about reproducing fly physiology. Keep useful market computation and biological fidelity as separate evaluation goals.

C. Test policy learning and plasticity separately

Later options include imitation of successful behaviors, offline reinforcement learning and explicit reward-modulated plasticity. Wallet actions do not reveal all information, hedges or objectives available to their owners, so copy data is not an unquestionable expert label. Historical trajectories also do not tell us the consequences of every unchosen action.

Only consider online learning in replay or shadow operation first. Demonstrate retention, stability and performance after regime changes. Stimulating a neuron labeled as dopaminergic does not automatically implement credit assignment or profitable learning.

Stages and deliverables

Stage	Work	Completion evidence
0. Research specification	Select one launchpad/factory and DEX destination; verify versions, data licensing, APIs, history and costs; define capital scenarios and train/test horizons	Written source/endpoint matrix; unresolved gaps explicit; proposed target and evaluation frozen before final holdout
1. Observation feasibility	Record all launches from the chosen source, including dead/failed tokens; join trades, liquidity and wallet observations	Coverage audit against chain logs; duplicate/reorg handling; measured arrival delay and missingness; versioned schema
2. Replay and baselines	Build complete entry-to-exit ledger; test watch-only, simple confirmation/momentum and conventional predictors	Reconciled sample transactions; stated fill assumptions; failures retained; costs and delays stressed
3. Neural representation test	Compare fixed fly reservoir with raw-feature readout, random reservoir, shuffled graph and compact conventional recurrent model	Chronological unseen evaluation; several seeds; matched tuning; paired uncertainty estimates; meaningful ablations
4. Hybrid tool/LLM experiment	Add LP-wallet features and then structured LLM features, one at a time	Marginal benefit, latency, source reliability and cost measured; results compared with the unchanged baseline
5. LP action experiment	Add selected ranges and LP-vs-spot-vs-idle decisions for supported pools	Fee attribution, tick behavior, hook effects and liquidation economics validated; no double-counting IL/LVR
6. Shadow operation	Record live hypothetical decisions using actual arrival times	Stable ingestion and decision latency; replay/live mismatch report; predefined observation period completed

Funded deployment is a separate later decision, based on evidence. No stage here authorizes it. Avoid promising calendar durations until data access and compute requirements are known.

Evaluation contract

- Primary financial outcome: net portfolio change in a declared numeraire, under fixed capital and exposure rules, measured against the strongest preselected baseline. Report absolute outcomes as well as excess outcomes.
- Report drawdown, tail losses, exit failures, turnover, capital utilization, prediction calibration, latency and costs. Do not optimize solely for win rate.

- Split chronologically; purge overlapping label horizons. Group related token/deployer episodes where appropriate to reduce cross-split leakage. Fit normalization and select models using training/validation only.
- Preserve launches that fail, never graduate, lose liquidity or become unsellable. Track unresolved outcomes explicitly; conservative valuation assumptions need sensitivity tests.
- Historical LLM enrichment must use only information available at the decision time. A present-day model may know subsequent outcomes, even from a token name. Prefer contemporaneous archived text, identity masking and a no-LLM baseline; use prospective shadow tests where leakage cannot be excluded.
- Model transaction delay, fees, slippage and failed exits. Candles alone cannot validate exact LP fees or executable fills in thin markets. Small-position assumptions do not establish scalability.
- Compare LP versus spot under matched initial capital and clearly stated inventory benchmarks. Do not subtract IL or LVR again if already represented in complete balance changes.
- Ablations: remove recurrent state, remove LP features, remove LLM features, randomize topology with relevant graph statistics matched, and freeze learning. Isolate which component contributes.

Go / revise / stop criteria

Advance only if the candidate beats both a strong practical baseline and a matched nonbiological model on the predeclared metric, with acceptable drawdown and robustness across unseen periods and realistic cost stress. Set numerical thresholds after a feasibility sample, before looking at the final holdout.

If recurrence helps but fly topology does not, retain the useful ordinary model and report the negative biological result. If no method has a robust net advantage, improve observations or revise the task; do not deploy because the narrative is appealing.

Training research update

[Training and hybrid AI design](#) now records the targeted source check performed after this initial plan was written. It identifies trainable parameters, feedback requirements, connectome constraints and LLM/tool interfaces. The next planning task is Stage 0's source/endpoint matrix and narrow dataset/target specification.

Foundational references and current tool evidence are in [the literature review](#) and [project direction](#). No new literature search was performed before writing this initial plan.

Historical literature review

HISTORICAL APPENDIX

This is the foundational review from the earlier LP-first research stage. Its scientific evidence and access notes are preserved. Its original recommendation to start with LP range selection has been superseded by the Observatory and launch-trading roadmap in the preceding sections.

Scope update - 13 September 2026: This is the original literature review. The project now prioritizes Robinhood Chain new launches, memecoins and spot-trading decisions, with selective LP as a separate action. Its LP-first recommendations below are historical. See [current direction](#), [experiment plan](#) and [build roadmap](#) for the active scope. The updated PDF places the active plan first and preserves this review as a historical appendix.

A feasible experiment, with an unproven financial advantage. The useful opportunity is to test whether connectivity measured from a fly nervous system improves a small, stateful controller for DEX liquidity decisions. The reviewed evidence does not establish profitable fly-brain liquidity provision, a complete digital animal, or a conscious uploaded mind.

FlyWire Codex is an explorer of reconstructed nervous-system data. It is one part of a stack: anatomical data, simulation equations, a learning rule, an input/output interface and an environment. Those parts must be chosen and validated separately. Public source code for Codex is available; the website itself is not a running brain.^{1,6}

The recommendation

Use the **FAFB/FlyWire dataset with the original Shiu implementation** as the scientific reference for reproducing published sensorimotor results. For an LP prototype intended to leave room for commercial use, prefer **MaleCNS v1.0 as the initial data candidate**, because its official release uses CC BY rather than the FAFB public release's noncommercial license. Porting between these datasets requires fresh validation.^{2,3,8}

The first experiment should ask one narrow question: **does a fly-derived recurrent network help choose liquidity ranges after realistic costs, beyond what the same inputs and ordinary models can do?** Start with historical replay and a fixed recurrent network plus a small trained output layer. Add online plasticity only after there is a measurable reason to do so. This is a proposed research design, not a result.

Can we download and use real fly wiring? | Yes, subject to the particular data license and release.

Can it support learning and control? | Yes, with explicitly engineered dynamics, training and interfaces.

Does that make it a real biological brain? | It makes a computational model using biological measurements.

Can it outperform ordinary LP strategies? | Unknown. The decisive market experiment has not been established by the reviewed evidence.

Scope. Literature and public resources were checked through 13 September 2026. This is a targeted deep review of foundational connectomics, simulation, vision, learning, embodiment, reservoir computing and LP economics. It does not claim to read every FlyWire-related paper. The bibliography distinguishes methods reviewed, abstract screening, inaccessible full text and unvalidated project disclosures. No simulations or financial backtests were run for this report.

1. The correct data and software

FAFB v783 | Adult female brain; 139,255 reconstructed neurons.^{1,10} | Published brain-model reference; FAFB data are CC BY-NC 4.0.²

MaleCNS v1.0 | Male brain plus ventral nerve cord; 166,700 neurons, 11,710 types.⁴ | Preferred data candidate for an extensible LP prototype; CC BY 4.0.^{3,39}

BANC v888 | Female brain plus nerve cord; 158,262 neurons shown in Codex.¹ | Useful later for distributed sensorimotor-control comparisons.²³

MANC / MAOL | Nerve cord only / right optic lobe only.¹ | Useful specialized datasets; neither is an entire brain.

The original female brain is the appropriate answer when someone means the widely reported 2024 FlyWire brain. The newer complete central-nervous-system releases broaden the choices. The MaleCNS site records a 2026 release and links its Cell paper; its publisher full text was not accessible during this review, so current dataset facts here come from the official release documentation.²⁴

Codex applies dataset-specific thresholds to weak connections. Displayed graph edges are not individual synapse counts or an unfiltered connectivity inventory.⁴⁰

What we actually need to download

For FAFB v783, the official Zenodo release offers an approximately 852 MB connection table and a 1.1 MB proofread-ID list. The 9.5 GB synapse table includes a broader population of segments and approximately 130 million contacts; that is not the same population as the roughly 50 million proofread-brain contacts commonly reported. A graph experiment generally does not need the underlying electron-microscopy image volume.^{5,10}

MaleCNS offers a 1.1 GB connection-weight table, 13 MB curated annotations and 42 MB transmitter predictions. The graph includes segments outside the curated neuronal set; filtering and ID joins are essential. Its bulk-download route avoids dependence on an interactive account, while the documented neuPrint API requires a token.³

Choose code independently from data

Codex: Apache-2.0 explorer code. **Shiu model:** MIT simulation code, with Brian2 and separate v630/v783 configurations; the default v630 path reproduces the paper. **Eon fly-brain:** accelerated simulation implementations, chiefly GPL-2.0-or-later with an upstream MIT exception, and Linux/NVIDIA-oriented GPU paths. Code licensing does not replace data licensing.^{6,8,9}

Recommendation: preserve an untouched paper-reference environment and create a separately named LP model. Keep a release manifest containing data URLs, hashes, licenses, filtering rules and software commits. An LP model using MaleCNS must not be presented as an exact reproduction of a FAFB experiment.

2. What "a real brain" would mean

A connectome records reconstructed anatomical connectivity from a biological specimen. It does not record a complete functioning state: moment-to-moment electrical activity, all effective synaptic strengths, receptor distributions, internal chemical state and learned dynamics are not supplied as a ready-to-resume nervous system. Whole-brain annotation also reveals both shared organization and differences between animals.^{10,11}

Biological origin | The connectivity was measured from actual fly tissue.

Biological tissue | A computer simulation contains mathematical state, not living cells.

Biological function | Specific responses can be tested against experiments. A correct response on one task does not validate all internal mechanisms.

Learning and autonomy | Possible engineered properties, requiring defined learning rules and an environment.

Conscious experience | Not demonstrated by neuron count, spiking, animal-like motion or trading performance.

Calling the system a "connectome-derived neural controller" is accurate. Calling it a complete uploaded fly would require evidence that substantially exceeds the studies reviewed here. This distinction does not make the experiment less useful: biological connectivity might be a valuable constraint even if its engineered dynamics differ from the original animal.

How this differs from an LLM

Our proposed controller would receive numerical observations, maintain a recurrent state and output a few constrained actions. It would have no pretrained language interface or stock of textual knowledge. We could test it without an LLM anywhere in the decision loop. Its learning would still be computational learning; biological inspiration does not place it outside artificial intelligence.

Consciousness and living-neuron alternatives

Butlin and colleagues propose assessing artificial systems using properties motivated by competing scientific theories of consciousness. That is a framework for investigation, not evidence that a FlyWire simulation has experience. The appropriate conclusion is **undetermined and unvalidated**, rather than certainty in either direction.²⁵

DishBrain is a different category: Kagan and colleagues coupled living cultured neurons to a simplified game environment. It is a wet-laboratory system with specialized interfaces, not a downloadable fly brain. The paper's use of "sentience" should not be treated as a settled demonstration of subjective experience. It supplies no evidence of DEX liquidity skill.²⁶

For this project, biological fidelity and financial usefulness should be evaluated on separate axes. A financially successful simplified reservoir would establish useful computation, not consciousness. A biologically informative simulation could be financially useless.

3. What the strongest neuroscience establishes

An anatomical map is powerful, but it is not a functional specification

The Dorkenwald reconstruction provides the foundational whole-adult-brain wiring resource. Schlegel and colleagues add extensive cell typing and comparisons between connectomes. The visual-system parts-list work organizes specialized visual circuitry. Together these provide experimentally grounded structure for selecting neurons, pathways and hypotheses; they do not assign those circuits a meaning such as "ETH price" or "profitable range."^{10,11,38}

Whole-brain spiking: Shiu et al., 2024

The Shiu model uses simplified leaky integrate-and-fire neurons to study selected sensorimotor pathways, including feeding-related inputs and grooming. The paper reports agreement for 91% of 164 empirically tested predictions. That denominator is a selected experimental set, not all fly behavior and certainly not financial decisions.⁷

Its limitations matter directly to adaptation: simplified cell dynamics and synaptic scaling, omitted physiological detail, uncertain neurotransmitter effects and a zero-activity baseline that can limit inhibitory effects. The published implementation does not supply a general plasticity mechanism. Reproducing its results is therefore a useful sanity check, not proof that a new learning controller is biologically faithful.^{7,8}

Vision: Lappalainen et al., 2024

This study combines visual-connectome constraints with task optimization to predict neuronal responses. Its model uses a specialized, tiled visual network of 64 cell types, with 45,669 model neurons and relatively few shared neural parameters. It is neither a generic visual language model nor a full FAFB brain wired directly to a camera. The released flyvis implementation is a relevant starting point for visual experiments.^{13,37}

A candlestick chart is a human encoding convention. Giving its pixels to simulated visual cells would introduce an additional learning problem and sensitivity to colors, scales and chart layout. For the first LP test, numerical features provide a more interpretable interface. A later chart experiment should randomize visual styling while preserving the underlying market series.

Chemical labels and effective influence remain uncertain

Eckstein and colleagues predict transmitter identity from electron-microscopy images; the reported accuracy differs at synapse, neuron and cell-type levels. These are predictions, not complete receptor-dependent synaptic effects.¹² Pospisil and colleagues distinguish anatomical connectivity from causal effective influence and propose experimental methods to infer the latter.¹⁴

Engineering implication: neurotransmitter-to-sign mappings, weight transformations and background activity must be explicit model assumptions. Test their sensitivity. Papers in this area do not all use the same sign or weight conventions, so mixing implementations can silently change the scientific hypothesis.

4. Memory, learning and useful computation

The closest evidence to a market application is reservoir computing

A reservoir is a recurrent dynamical system whose changing activity summarizes recent inputs. A small trained readout converts that activity into predictions or actions. Keeping recurrent weights fixed makes the first experiment easier to interpret: any advantage can be investigated without simultaneously changing the entire graph.

Costi and colleagues tested fly-connectome-derived echo-state networks on orbital time series from a restricted three-body problem. They examined smaller subsets and a filtered full network. Some settings showed resistance to overfitting, but a random reservoir performed better under stronger regularization; weight distributions appeared more influential than topology in important comparisons. This is evidence for a testable computational prior, not market forecasting or universal superiority.¹⁶

Earlier Morra-Daley studies also investigate connectome-constrained echo-state networks and report conditional results across synthetic dynamical tasks.^{17,18} A 2026 optimization preprint uses a *Drosophila* representation with only 49 nodes. Its title should not be read as evidence about a full adult fly brain or financial performance.³⁶

Three different kinds of memory

Recurrent state | Activity while weights stay fixed | How long past observations remain recoverable; benefit over a state-reset control.

Readout learning | Parameters that map activity to actions | Held-out prediction or net LP performance after a fixed training period.

Synaptic plasticity | Connections or their effective strengths | Improvement from feedback, retention, interference and adaptation after regime changes.

Learning-circuit research in the larval mushroom body shows recurrent organization around dopamine-related teaching pathways. That gives biological motivation for studying structured feedback; it does not establish that an adult whole-brain simulator learns automatically when a "dopamine" input is stimulated.¹⁵

Why profit-as-reward is not enough

A learning rule must specify which connections change, their eligibility traces, the delayed feedback signal, learning rates and stability constraints. LP rewards are delayed and confounded by price exposure and unrelated market moves. A positive portfolio return can occur despite a poor liquidity decision. Directly labeling gains "sugar" and losses "bitter" supplies a stimulus mapping, not causal credit assignment.

Recommendation: begin with a frozen recurrent network and supervised readout. If it adds measurable value, test reward-modulated plasticity as a separate intervention, against frozen-learning and shuffled-reward controls. A learned market task would then be an engineered capability that we can measure, rather than an assumed property of the source animal.

5. Embodiment: what moving flies demonstrate

A convincing animated animal can combine several systems: a physics model, trained motor policies, sensory preprocessing and some connectome-derived activity. We need to identify which component controls each behavior before attributing the result to the reconstructed nervous system.

Peer-reviewed body platforms

NeuroMechFly v2 provides a body-and-environment framework with vision, olfaction, motor feedback and challenging terrain. Its demonstrations include designed and reinforcement-learned controllers. It is useful for testing closed-loop hypotheses, but a body platform alone does not provide a complete brain model.¹⁹

The flybody work produces realistic walking and flight using whole-body physics and trained neural controllers. Its use of learned control is a concrete reason that natural-looking movement cannot by itself identify the biological mechanism generating it.²⁰

Whole-connectome control models and company demonstrations

FlyGM, a 2026 preprint, places trainable neural components around a connectomic graph and uses expert imitation followed by reinforcement learning for locomotion. Its comparisons do not cleanly isolate topology alone: biological weighted/signed connections and some unweighted control graphs differ in more than wiring. The result is an interesting trained architecture, not evidence that an untouched connectome already contains a working digital animal.²¹

Eon's March 2026 technical disclosure describes a hybrid system using existing brain, vision and body components with selected interfaces and motor controllers. It acknowledges substantial missing learning and internal-state mechanisms, limited behavioral influence of the implemented visual pathway, and important unvalidated neural dynamics. This disclosure is more informative than treating a "brain upload" headline as an established scientific conclusion.²²

What the newer brain-and-cord data change

The BANC study maps distributed control circuitry across brain and nerve cord. That improves the anatomical basis for studying interactions between descending commands, ascending feedback and local motor circuits. It does not automatically solve the dynamics of those interactions.²³ MaleCNS similarly supplies a complete central-nervous-system data resource for new modeling work.²⁴

For an LP agent, a simulated fly body is optional. Its environment can be a pool state and portfolio; its actions can be bounded liquidity adjustments. This would be computational embodiment in a financial task, not a faithful reproduction of fly locomotion. A 3D fly visualization may help explain activity, but should never serve as evidence that the policy learned anything.

The most valuable later body experiment would ask whether a specific connectome-based control hypothesis reproduces both behavior and relevant neuronal activity under perturbation. Financial tests should instead evaluate decision quality and accounting. Keeping those targets separate prevents a visual demonstration from standing in for either form of validation.

6. What a DEX liquidity controller could do

The smart contract already performs automated market making. Our experimental controller would decide how to allocate a portfolio around it: whether to supply liquidity, where to place a range, how wide it should be, how much to deploy and when to adjust. It would not need to understand a protocol in language to select these actions.

Uniswap v3 permits bounded liquidity ranges. Fees accrue while a position participates in active liquidity; outside its interval it becomes one-sided and stops earning swap fees until price returns. Narrow ranges concentrate exposure as well as fee participation. Fee allocation depends on active liquidity along the swap path, not simply the pool's total value locked.²⁷

The scientific opportunity

A recurrent controller might recognize changing volatility, persistent flow or a rising probability of leaving a range. These are hypotheses. Its useful role could be selecting a conservative range or avoiding an unfavorable interval, rather than forecasting the next token price. A transparent numeric interface also lets us determine which observations matter.

Range width | Trailing volatility, recent range exits, liquidity concentration and estimated adjustment cost.

Recenter or wait | Distance to range boundaries, current inventory, expected fee opportunity and transaction delay.

Deploy or remain idle | Available capital, recent external/pool price divergence, flow conditions and risk limits.

Fees are not the same as investment skill

The LVR literature separates market exposure from the economics of supplying liquidity against better-informed arbitrage. Loss-versus-rebalancing compares with a dynamic benchmark; loss-versus-holding compares with retaining initial assets. These are different counterfactuals. Neither is a generic extra invoice to subtract twice from an already complete portfolio ledger.²⁸

Fritsch and Canidio find historical pools where estimated arbitrage losses exceeded fee earnings. That is evidence that high activity can coexist with unattractive liquidity economics, not a statement that every present-day pool loses money.²⁹

Online-learning LP research offers a useful simpler benchmark, although the Bar-On-Mansour formulation adopts simplifying assumptions such as omitted gas costs.³⁰ The Zhang-Chen-Yang reinforcement-learning study reports simulated LP improvements and includes a hedging formulation, but assumes flat adjustment gas, frictionless hedging, no adjustment slippage and no impact on other traders. Its results are not live profitability evidence.³¹

Recommendation: use one historical WETH/USDC Uniswap v3 pool as a research case after data-quality checks. Keep futures hedging out of the first experiment so that the contribution from liquidity decisions is easier to identify. This is a proposed benchmark environment, not a recommendation to deposit funds.

7. Existing "fly traders" and our proposed design

Public prior art exists, but it does not settle the question

Stonkfly connects a MaleCNS-derived simulation to a spot-trading environment. Its documentation describes chart-based input, a fixed trading readout and candidate reward-driven plasticity. The authors explicitly state that profitable learning has not been demonstrated. This is relevant implementation prior art, not a verified DEX LP result.^{32,33}

Traderfly-brain describes market features converted through predetermined thresholds into sensory stimuli, with biological outputs mapped to trading decisions. It restarts activity for decisions and keeps parts of the surrounding strategy private. Its repository states all rights reserved, so public visibility should not be confused with an open-source license. It is not evidence of a learned, persistent LP controller.³⁴

The research gap is therefore specific: establish whether connectivity itself adds reproducible, cost-adjusted value after controlling for the engineered features, readout and surrounding strategy. A profitable wrapper could conceal a neural component that contributes little or nothing.

Proposed architecture

1. Market observations | Only timestamped numeric data available before the action: price changes, flow, volatility, active liquidity and costs.
2. Fixed input encoder | A documented mapping from standardized observations to model inputs. No hand-coded buy/sell or narrow/wide recommendation hidden inside it.
3. Recurrent network | Sparse fly-derived graph with explicitly chosen dynamics. Preserve state across observations within an episode.
4. Small trained readout | Predict range-exit probabilities or select among a few permitted liquidity actions.
5. Deterministic controls | Enforce maximum allocation, minimum holding time, transaction budgets and allowed ticks.
6. Historical pool replay | Account for balances, fees, transactions and delayed execution; return observations and evaluable outcomes.

Initially fix the range center at the observed pool price when entering, use a constant deployment fraction, and let the model choose only width or remaining idle. This isolates a manageable question. Add range-center shifts, variable capital and continuous actions only if the restricted policy demonstrates value.

The first task can predict whether price leaves each candidate range over a fixed future horizon. Every model then uses the same deterministic range-selection rule. This separates a representation benefit from a complicated policy-training benefit. Direct reward optimization is a later experiment, not a prerequisite for learning something useful.

8. A test that can produce a credible answer

Pre-register two hypotheses

H1: useful temporal representation. With identical observations and training budgets, a fly-derived reservoir predicts range exits or volatility better on unseen market periods than a readout without the reservoir. **H2: useful liquidity decisions.** Those improvements translate into better net portfolio outcomes than strong simple LP strategies, at comparable exposure and drawdown. H1 can succeed while H2 fails.

Hold initial assets; cash reference | Whether apparent gains mainly follow asset prices or a change in exposure.

Fixed wide-range LP | Whether adaptation adds anything beyond passive liquidity.

Volatility-based range rule; online learner | Whether a straightforward adaptive strategy is already sufficient.

Readout on the same raw features | Whether engineered observations explain the result without recurrence.

Ordinary random reservoir | Whether recurrence helps but biological structure is unnecessary.

Degree/sign/weight-controlled shuffled graph | Whether biological topology helps after other graph properties are controlled.

Conventional small recurrent model | Whether a standard trained model uses the available data and compute more effectively.

Chronological evaluation

Use rolling training and validation windows followed by untouched test windows. Freeze normalization, feature choices, hyperparameters and action definitions before each test. Separate windows by the prediction horizon where overlapping labels would leak information. Preserve a final holdout that is not inspected while deciding the model. Report every pre-specified market period, including adverse ones.

Repeat initialization and input-mapping seeds. These measure training sensitivity, not independent financial histories. Estimate uncertainty across suitable time blocks or market episodes, and compare strategies on the same periods. Match both tuning opportunities and practical computation; matching neuron counts alone is not a fair comparison.

Interventions that expose a decorative brain

Reset recurrent state, shuffle input-channel assignments, freeze learning, shuffle rewards, and perturb graph organization while matching key degree, sign and weight statistics. Where feasible, also match stability scaling. If performance survives removal of the neural component, attribute it to the surviving system rather than to biological wiring. For a plasticity claim, verify actual parameter changes and retention on a later test.

A promising result should improve the pre-specified primary metric against both a strong simple LP baseline and a matched nonbiological network, remain useful across multiple unseen periods, and survive realistic cost stress. If the uncertainty interval remains wide or the advantage disappears with modest costs, the answer is inconclusive or negative. Do not keep changing the metric until a positive result appears.

9. The accounting is part of the experiment

Use an event-based replay where practical: swaps, liquidity additions/removals, ticks, active liquidity and fee growth, together with block timestamps and an external reference price. Candles can support an initial prediction task, but do not contain enough information for precise path-dependent fee attribution. The replay should be validated on simple positions before any neural policy is evaluated.²⁷

Maintain one complete portfolio ledger

Define equity as the marked value of every token balance, active position, idle holding and uncollected fee. Account for gas, swap fees, slippage and execution delay when they occur. Record rejected or failed actions. Withdrawing a position leaves its underlying token inventory; becoming "idle" does not automatically turn that inventory into stablecoin.

A possible training objective is **change in net equity minus change in a stated benchmark, normalized by starting capital**, with optional pre-declared drawdown or inventory penalties. Evaluate the unpenalized financial ledger as well. The penalty is an engineering preference, not a measured economic expense.

Do not subtract "impermanent loss" and "LVR" as additional costs from a ledger that already contains all token-value changes. Report those concepts separately, with their respective holding and rebalancing benchmarks. A model must not appear profitable merely because a diagnostic loss was omitted or counted inconsistently.²⁸

Counterfactual and execution limitations

Historical swaps occurred under historical liquidity. Inserting our LP changes fee shares and potentially price paths and trader behavior. Begin with an explicitly small, price-taking hypothetical position; recalculate its local fee participation and state the unmodeled feedback. Larger deployments require a more demanding impact model. The small-position assumption is a limit of the experiment, not proof of scalability.

Use realistic tick rounding, required token ratios, rebalance timing and transaction costs. Test delayed execution and higher cost scenarios. Include external-price dislocations, stablecoin deviations and periods with poor fee opportunity where present in the data. Treat additional transaction-ordering or MEV costs as explicit modeled assumptions if historical data cannot identify them.

Metrics to publish together

Net excess return against the stated baseline | Absolute equity return; maximum drawdown; distribution of period returns.

Paired advantage over adaptive LP controls | Token exposure; fees; transaction costs; turnover; fraction of time active.

Stability under cost and delay stress | Results by market regime and seed; prediction calibration; reward and graph ablations.

An attractive annualized return from one favorable interval is not sufficient. The experiment should produce a reproducible decision ledger so that every action can be traced to past inputs and every gain can be traced to inventory changes or earned fees.

10. Implementation path and further experiments

A staged path with explicit stopping points

Stage 1: reference and data integrity. Reproduce a small published Shiu stimulus-response example in its own environment. Separately ingest the selected LP graph, validate IDs, signs, duplicate aggregation and matrix orientation. Do not claim that running a new dataset reproduces a paper's old dataset.

Stage 2: inexpensive prediction benchmark. Start with a documented subnetwork of roughly 1,000 nodes, then test a larger one if justified. Measure out-of-sample representation value before building an expensive whole-network training system. A subnetwork is an engineering approximation and must be labeled as such.

Stage 3: LP replay. Validate the accounting independently, compare restricted range policies and run the interventions described above. **Stage 4: shadow operation.** If replay results remain convincing, observe live inputs and record hypothetical actions to test latency and data reliability. Any funded deployment would be a separate decision after those results.

Practical computational choices

Store graphs sparsely. A dense 139,255 by 139,255 float32 matrix alone would require approximately 77.6 GB in decimal units, before simulation state or training memory. Preserve neuron IDs as integers; never route large IDs through floating-point values. Document whether matrix rows represent receiving or sending neurons. Estimate actual memory from the filtered edge set.

The original Brian2 code is a portable starting point for reference experiments. Accelerated implementations may introduce operating-system and hardware constraints.^{8,9} A Loihi 2 preprint demonstrates another hardware route for large connectome simulation, but does not establish learning, consciousness or a financial advantage.³⁵ No runtime or hardware budget has been benchmarked for this project.

Biological milliseconds and market minutes are not automatically equivalent. Input presentation duration, update frequency, state decay and learning timescale are experiment parameters. Fix and test them. For large reservoirs, avoid accidentally constructing dense training matrices or inverses that erase the memory benefit of sparse simulation.

Other worthwhile experiments

Regime-shift detection | Does recurrence detect changing conditions earlier than an ordinary filter?

Learning and forgetting | Does explicit plasticity adapt and retain useful behavior without excessive interference?

Circuit perturbation | Are identifiable graph components necessary for the measured computation?

Embodied navigation | Do specified circuits predict both movement and neural responses under perturbation?

The recommended next discussion is to choose the main objective: financial performance, learning mechanisms or biological fidelity. For the stated DEX goal, start with the frozen-reservoir range experiment. Its value is that a positive, negative or inconclusive result can each be scientifically informative.

Sources and reading coverage

1. [FlyWire Codex](#). Official dataset explorer. Dataset catalogue inspected; current release names and displayed counts.
2. [FlyWire public-data guidelines](#). Official release terms. Public-release and license statements inspected.
3. [MaleCNS: download the dataset](#). Official data documentation. File descriptions, filtering context, API and CC-BY license inspected.
4. [MaleCNS media gallery](#). Official dataset description. Neuron and cell-type counts verified.
5. [FlyWire whole-brain connectome, v783.0](#). Zenodo, 2024. File manifests, sizes and release descriptions inspected.
6. [murthylab/codex](#). Official source repository. README and Apache-2.0 license inspected; software not executed.
7. [Shiu et al. A Drosophila computational brain model reveals sensorimotor processing](#). Nature, 2024. Main results, selected methods and limitations reviewed.
8. [philshiu/Drosophila_brain_model](#). Paper implementation. README, release configuration and MIT license inspected; not executed.
9. [eonsystemspbc/fly-brain](#). Simulation software repository. README, hardware requirements and licensing inspected; not benchmarked.
10. [Dorkenwald et al. Neuronal wiring diagram of an adult brain](#). Nature, 2024. Abstract and accessible main-text excerpts reviewed; full-page access was intermittent.
11. [Schlegel et al. Whole-brain annotation and multi-connectome cell typing of Drosophila](#). Nature, 2024. Main text, selected methods and data availability reviewed.
12. [Eckstein et al. Neurotransmitter classification from electron microscopy images at synaptic sites in Drosophila melanogaster](#). Cell, 2024. Abstract and accessible results screened; not a full-methods audit.
13. [Lappalainen et al. Connectome-constrained networks predict neural activity across the fly visual system](#). Nature, 2024. Main results and model-construction methods reviewed.
14. [Pospisil et al. The fly connectome reveals a path to the effectome](#). Nature, 2024. Accessible main-text excerpts and causal-inference proposal reviewed.
15. [Eschbach et al. Recurrent architecture for adaptive regulation of learning in the insect brain](#). Nature Neuroscience, 2020. Abstract screened; larval learning-circuit evidence, not adult whole-brain validation.
16. [Costi et al. The Drosophila Connectome as a Computational Reservoir for Time-Series Prediction](#). Biomimetics, 2025; 10(5):341. Full accessible article reviewed, including methods, comparisons and limitations.
17. [Morra and Daley. Imposing Connectome-Derived Topology on an Echo State Network](#). arXiv:2201.09359, 2022. Abstract screened; earlier connectome-derived reservoir work.
18. [Morra and Daley. Using Connectome Features to Constrain Echo State Networks](#). arXiv:2206.02094v2, revised 2023. Abstract screened; conditional benefits and adverse topology modifications.
19. [Wang-Chen et al. NeuroMechFly v2: simulating embodied sensorimotor control in adult Drosophila](#). Nature Methods, 2024. Abstract, figure descriptions and code/data availability reviewed; subscription body unavailable.
20. [Vaxenburg et al. Whole-body physics simulation of fruit fly locomotion](#). Nature, 2025. Accessible main-text results and controller description reviewed.
21. [Jin et al. Whole-Brain Connectomic Graph Model Enables Whole-Body Locomotion Control in Fruit Fly](#). arXiv:2602.17997v1, 2026; preprint. Methods, training pipeline and comparison tables reviewed.

22. [Eon Systems. Embodied brain emulation: technical disclosure](#). Company technical report, March 2026. Full disclosure reviewed; company demonstration, not independent validation.
23. [Bates et al. Distributed control circuits across a brain-and-cord connectome](#). Nature, 2026. Abstract and selected main-text passages reviewed.
24. [Male CNS Connectome](#). Official release site, 2026. Release information and paper links inspected; Cell full text not accessible.
25. [Butlin et al. Consciousness in Artificial Intelligence: Insights from the Science of Consciousness](#). arXiv:2308.08708v3, 2023. Abstract and indicator-based framing screened; not a consciousness test of FlyWire.
26. [Kagan et al. In vitro neurons learn and exhibit sentience when embodied in a simulated game-world](#). Neuron, 2022. Primary-paper summary and experimental setup screened; living cultured neurons.
27. [Adams et al. Uniswap v3 Core](#). Protocol whitepaper, 2021. Concentrated-liquidity mechanics and fee-accounting sections reviewed.
28. [Milionis et al. Automated Market Making and Loss-Versus-Rebalancing](#). arXiv:2208.06046v6, 2026 revision of 2022 work. Return decomposition, assumptions and LVR/LVH distinction reviewed.
29. [Fritsch and Canidio. Measuring Arbitrage Losses and Profitability of AMM Liquidity](#). ACM Web Conference Companion, 2024. Abstract, empirical setup and discussion/conclusion reviewed.
30. [Bar-On and Mansour. Uniswap Liquidity Provision: An Online Learning Approach](#). arXiv:2302.00610v2, 2023. Learning formulation and simplifying assumptions reviewed.
31. [Zhang, Chen and Yang. Adaptive Liquidity Provision in Uniswap V3 with Deep Reinforcement Learning](#). arXiv:2309.10129v1, 2023; preprint. Reward, simulation assumptions, baselines and evaluation setup reviewed.
32. [nftechie/stonkfly](#). Public experimental repository. README and stated limitations inspected; no independent performance verification.
33. [Stonkfly: model documentation](#). Project technical disclosure. Encoder, decoder, learning claims and limitations reviewed.
34. [SotoAlt/traderfly-brain](#). Public project repository. README, decision pipeline and rights statement inspected; not executed.
35. [Neuromorphic Simulation of Drosophila Melanogaster Brain Connectome on Loihi 2](#). arXiv:2508.16792, 2025; preprint. Abstract screened; hardware mapping evidence only.
36. [Guragain et al. The Whale That Outswam Evolution: Swarm Intelligence Maximises Memory in Connectome Reservoirs](#). arXiv:2606.09902, 2026; preprint. Abstract screened; the Drosophila representation contains only 49 nodes.
37. [TuragaLab/flyvis](#). Visual-model source repository. Official implementation identified; not executed.
38. [Matsliah, Yu et al. Neuronal parts list and wiring diagram for a visual system](#). Nature, 2024. Accessible main-text and data-availability passages reviewed.
39. [Creative Commons Attribution 4.0 International](#). Official license deed. License permissions and attribution conditions inspected.
40. [Codex frequently asked questions](#). Official documentation. Connection thresholds, data interpretation and access guidance inspected.